

Constraint Satisfaction Problems

בעיית CSP מורכבת משלשה $\langle V, D, C \rangle$ כאשר:

V אוסף סופי של משתנים $V = \{X_1, \dots, X_n\}$

D הדומיינים $D = \{D_1, \dots, D_n\}$ שמהם המשתנים מקבלים ערכים

C אוסף של אילוצים

לדוגמה - סודוקו:

- המשתנים V הם התאים

- הדומיינים D הם הספרות 1 עד 9. אבל:

- אם כבר יש ערך k בתא - הדומיין של התא הוא $\{k\}$.

- אם התא ריק - הדומיין הוא $\{1, \dots, 9\} \setminus \{\text{conflicting cell's digits}\}$ (לפי הבעיה הראשונית)

- האילוצים - AllDifferent

אפשר לפתור CSP בשתי שיטות:

- חיפוש סיסטמטי - עובר על כל המצבים - קוטס רק אם הוא יכול להוכיח שהוא הגיע למצב שאי אפשר להגיע ממנו לפתרון.

- חיפוש לוקאלי - מתחילים מהצבה אקראית, ומחפשים בכל איטרציה את השכן שמקדם אותנו. יכול להיתקע במינימום מקומי.

חיפוש לוקאלי

חיפוש לוקאלי מבצעים באמצעות iterative repair.

- הרעיון: מתחילים מהשמה אקראית.

- בכל איטרציה:

- בוחרים משתנה אקראי שמפר אילוצים.

- משנים את הערך שלו כדי שיפחית את מספר האילוצים שנפגעים.

- עוצרים כאשר אין יותר הפרות - או אחרי step limit.

בגלל שיש step limit, האלגוריתם לא שלם. אבל הוא מאוד מהיר.

חיפוש סיסטמטי

- הרעיון: עובדים עם השמה חלקית (בניגוד לחיפוש לוקאלי, שם אנחנו תמיד בהשמה מלאה)

- נכנסים ברקורסיה¹ לכל תוספת להשמה.

¹לא לעשות רקורסיה - עדיף לעבוד עם לולאות.

• כשמגיעים לקונפליקט באילוצים, עושים backtracking.

הסיבוכיות של זה אקונומציאלית, וכדי לשפר את זה נרצה לעשות informed backtracking. אפשר לעשות את זה עם יוריסטיקות ועם propogation.

יוריסטיקות

היוריסטיקה הראשונה נקראת MRV - Minimum Remaining Values. כשצריך לבחור משתנה לפתח איתו את העץ, בוחרים את המשתנה עם הכי מעט אפשרויות (ממה שנשאר עד כה, לפי האילוצים):

$$x^* = \arg_{x \in V} \min_{\text{unassigned}} |D(x)|$$

הרעיון הוא שהמשתנה הזה יתן לנו branching קטן, ואם אנחנו בתת-עץ ללא מוצא אז ככה ניכשל יותר מהר.

אם יש שוויון, אז מפעילים בתור tie-breaker יוריסטיקה שנקראת Degree heuristic - בוחרים את המשתנה שמשפיע על הכי הרבה משתנים אחרים. הרעיון הוא שהצבות אליו יגרמו לנו להיכשל יותר מהר כי הוא יקטין את remaining values של יותר משתנים אחרים.

אחרי שבחרנו משתנה, בשביל לבחור את הערכים נשתמש ביוריסטיקה שנקראת LCV - Least Constraining Value - שבוחרת ערך שפחות פוסל את האופציות שיש לשכנים. בניגוד לMRV - שמנסה להיכשל מהר - LCV דווקא מנסה להצליח. מכיוון שכבר בחרנו משתנה, וממילא אנחנו צריכים לעבור על כל הערכים שלו, נעדיף לתעדף את הערכים שיש להם סיכוי טוב כדי שאולי נגיע למטרה ונוכל להפסיק לחפש.

Propogation

אחרי שהצבנו ערך למשתנה, יכול להיות שכבר פגענו במשתנה שנראה רק בעתיד הרחוק. זה אומר שאין פתרון מכאן, אבל עדיין נצטרך להמשיך לפתח את העץ עד שנגיע למשתנה שמשתתף בקונפליקט. נרצה, לכן, לעשות pruning.

Forward Checking

אחרי שעושים הצבה, נעבור על כל השכנים. בנוסף לבדיקת constraints הרגילה, אם לשכן עדיין אין הצבה - נעדכן את domain שלו. אם הוא ריק - זה אומר שההצבה שלנו לא טובה, וצריך לעשות לה pruning.

(Arc Consistency) AC-3

אחרי כל הצבה נרצה לעבור על כל האילוצים ולבדוק אלו מהם אינם חוקיים - כלומר אי אפשר לעשות להם הצבה חוקית. AC-3 עובד רק על אילוצים בינאריים - כלומר כאלה שמשתתפים בהם 2 משתנים.

הרעיון: לכל קשת (אילוץ) $X_i \rightarrow X_j$, מסירים מ- $D(X_i)$ כל $a \in D(X_i)$ שלא קיים עבורו $b \in D(X_j)$ המקיים $C_{ij}(a, b)$

בינה מלאכותית
89-5570-10

מקליד: עידן אריה
מתרגל: דוד יצחקי
תאריך: 2026-06-18

אם מגיעים לשלב שבו $D(X)$ כלשהו הוא קבוצה ריקה - אזי אין פתרון משם, וקוטמים את הענף.

מתחילים מהמשתנה X_i שעשינו לו הצבה, ובכל פעם שמעדכנים $D(X_j)$ אז צריך גם לבדוק את השכנים של X_j - וכן הלאה.

AC-3 הוא מאוד יקר, ולכן יכול להיות שלא נפעיל אותו אחרי כל עדכון.